ATNLP: Lecture 13
Reasoning in LLMs

Shay Cohen

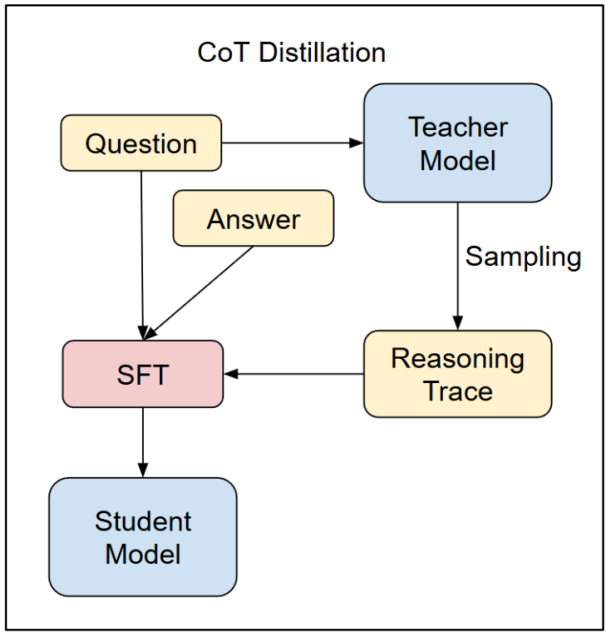
February 9, 2026

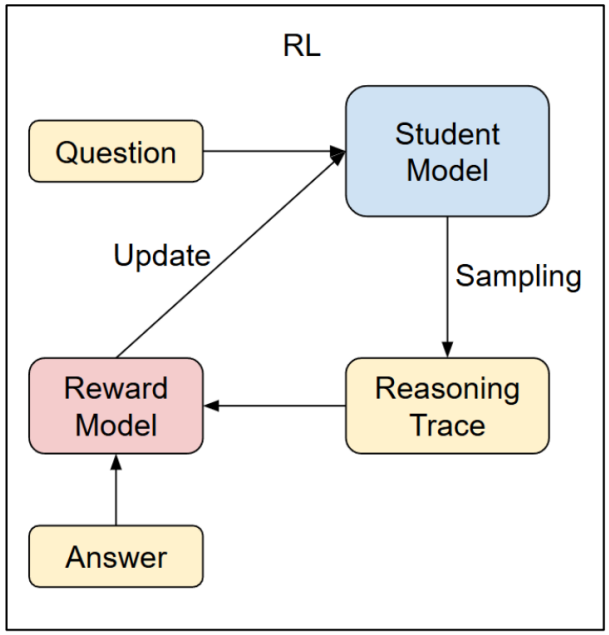
What is Reasoning?

ChatGPT says: To reason is to think in a structured way, using facts or assumptions to draw conclusions, make decisions, or explain why something follows from something else.

We will focus on the notion of using "text" (chain-of-thought, for example) to argue about/solve something that humans typically indeed need reasoning for

Reasoning - training





(Thanks to Shiwen Qin for figure)

- Two "training" approaches mostly (does the one on the left look familiar?)
- We will focus today on improving inference (easier within limited-budget settings)

Reasoning - inference

Food for thought: What are some ways to improve the reasoning of LLMs through inference?

Reasoning - inference

We will cover a few types of approaches:

- Stepwise: chain-of-thought, self-consistency, tree-of-thoughts...
- Self-reflection
- Using external solvers

The area is a moving target and has been developing very quickly

Today's Focus

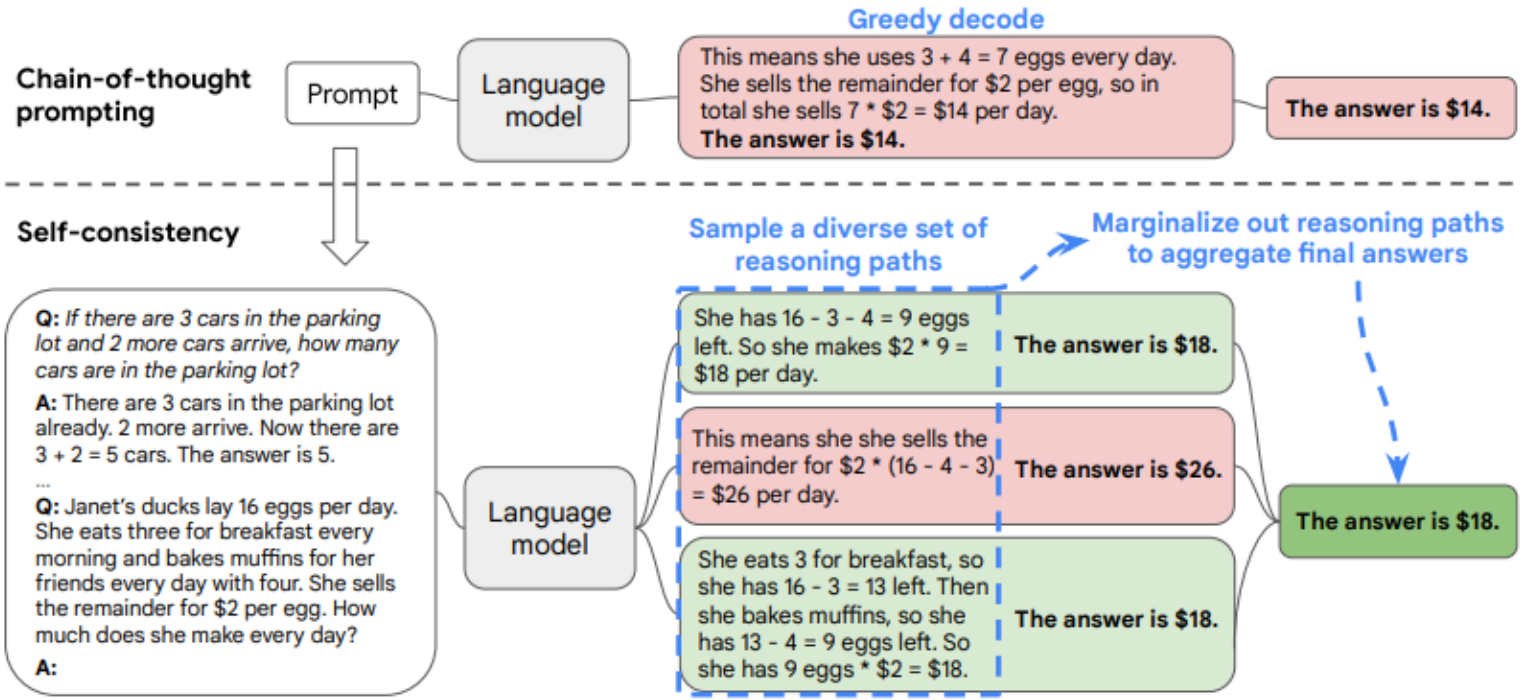
We will focus mostly on techniques that are follow at least some of the criteria:

- Simple and straightforward to conceptually understand and implement
- Mostly address the "decoding" phase – meaning, they are applied during inference (not training)

Self-Consistency

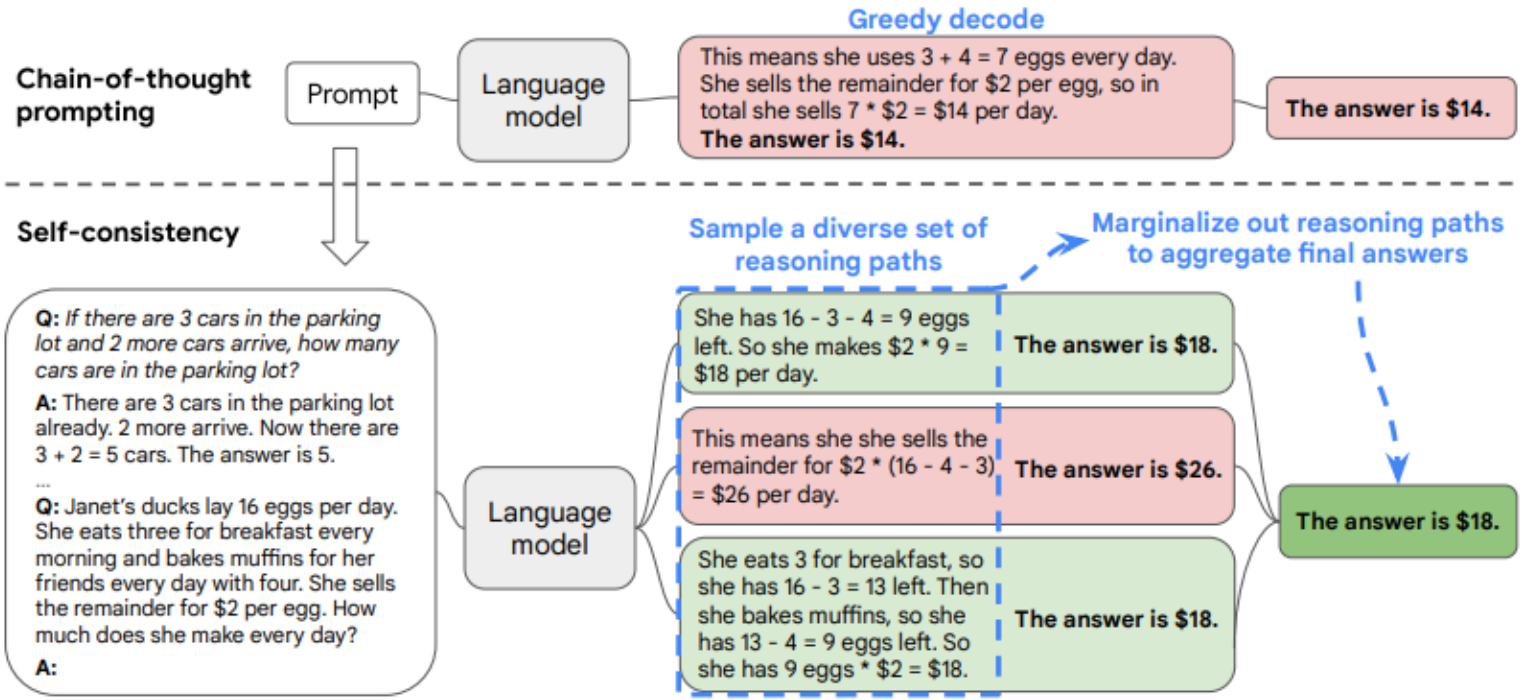
- The idea: there are different ways to reason about a problem. If several reasoning paths lead to the same answer, choose that answer
- How do we action that?
- Sample a set of reasoning chains $r_1, \ldots, r_n$ with answers $a_1, \ldots, a_n$
- Choose the answer a which repeats the most in the sequence a_i

The Basic Process



Food for thought: What is the simplest way to control the diversity of the answers?

The Basic Process



Food for thought: What is the simplest way to control the diversity of the answers? Changing the temperature.

Results

	Method	AddSub	MultiArith	ASDiv	AQuA	SVAMP	GSM8K
	Previous SoTA	**94.9**[a]	60.5[a]	75.3[b]	37.9[c]	57.4[d]	35[e] / 55[g]
UL2-20B	CoT-prompting	18.2	10.7	16.9	23.6	12.6	4.1
	Self-consistency	24.8 (+6.6)	15.0 (+4.3)	21.5 (+4.6)	26.9 (+3.3)	19.4 (+6.8)	7.3 (+3.2)
LaMDA-137B	CoT-prompting	52.9	51.8	49.0	17.7	38.9	17.1
	Self-consistency	63.5 (+10.6)	75.7 (+23.9)	58.2 (+9.2)	26.8 (+9.1)	53.3 (+14.4)	27.7 (+10.6)
PaLM-540B	CoT-prompting	91.9	94.7	74.0	35.8	79.0	56.5
	Self-consistency	93.7 (+1.8)	99.3 (+4.6)	81.9 (+7.9)	48.3 (+12.5)	86.6 (+7.6)	74.4 (+17.9)
GPT-3 Code-davinci-001	CoT-prompting	57.2	59.5	52.7	18.9	39.8	14.6
	Self-consistency	67.8 (+10.6)	82.7 (+23.2)	61.9 (+9.2)	25.6 (+6.7)	54.5 (+14.7)	23.4 (+8.8)
GPT-3 Code-davinci-002	CoT-prompting	89.4	96.2	80.1	39.8	75.8	60.1
	Self-consistency	91.6 (+2.2)	**100.0** (+3.8)	**87.8** (+7.6)	**52.0** (+12.2)	**86.8** (+11.0)	**78.0** (+17.9)

Table 2: Arithmetic reasoning accuracy by self-consistency compared to chain-of-thought prompting (Wei et al., 2022). The previous SoTA baselines are obtained from: a: Relevance and LCA operation classifier (Roy & Roth, 2015), b: Lan et al. (2021), c: Amini et al. (2019), d: Pi et al. (2022), e: GPT-3 175B finetuned with 7.5k examples (Cobbe et al., 2021), g: GPT-3 175B finetuned plus an additional 175B verifier (Cobbe et al., 2021). The best performance for each task is shown in bold.

Results

	Method	CSQA	StrategyQA	ARC-e	ARC-c	Letter (4)	Coinflip (4)
	Previous SoTA	**91.2**[a]	73.9[b]	86.4[c]	75.0[c]	N/A	N/A
UL2-20B	CoT-prompting	51.4	53.3	61.6	42.9	0.0	50.4
	Self-consistency	55.7 (+4.3)	54.9 (+1.6)	69.8 (+8.2)	49.5 (+6.8)	0.0 (+0.0)	50.5 (+0.1)
LaMDA-137B	CoT-prompting	57.9	65.4	75.3	55.1	8.2	72.4
	Self-consistency	63.1 (+5.2)	67.8 (+2.4)	79.3 (+4.0)	59.8 (+4.7)	8.2 (+0.0)	73.5 (+1.1)
PaLM-540B	CoT-prompting	79.0	75.3	95.3	85.2	65.8	88.2
	Self-consistency	80.7 (+1.7)	**81.6** (+6.3)	**96.4** (+1.1)	**88.7** (+3.5)	70.8 (+5.0)	91.2 (+3.0)
GPT-3 Code-davinci-001	CoT-prompting	46.6	56.7	63.1	43.1	7.8	71.4
	Self-consistency	54.9 (+8.3)	61.7 (+5.0)	72.1 (+9.0)	53.7 (+10.6)	10.0 (+2.2)	75.9 (+4.5)
GPT-3 Code-davinci-002	CoT-prompting	79.0	73.4	94.0	83.6	70.4	99.0
	Self-consistency	81.5 (+2.5)	79.8 (+6.4)	96.0 (+2.0)	87.5 (+3.9)	**73.4** (+3.0)	**99.5** (+0.5)

Table 3: Commonsense and symbolic reasoning accuracy by self-consistency compared to chain-of-thought prompting (Wei et al., 2022). The previous SoTA baselines are obtained from: *a*: DeBERTaV3-large + KEAR (Xu et al., 2021b), *b*: Chowdhery et al. (2022), *c*: UnifiedQA-FT (Khashabi et al., 2020). The best performance for each task is shown in bold.

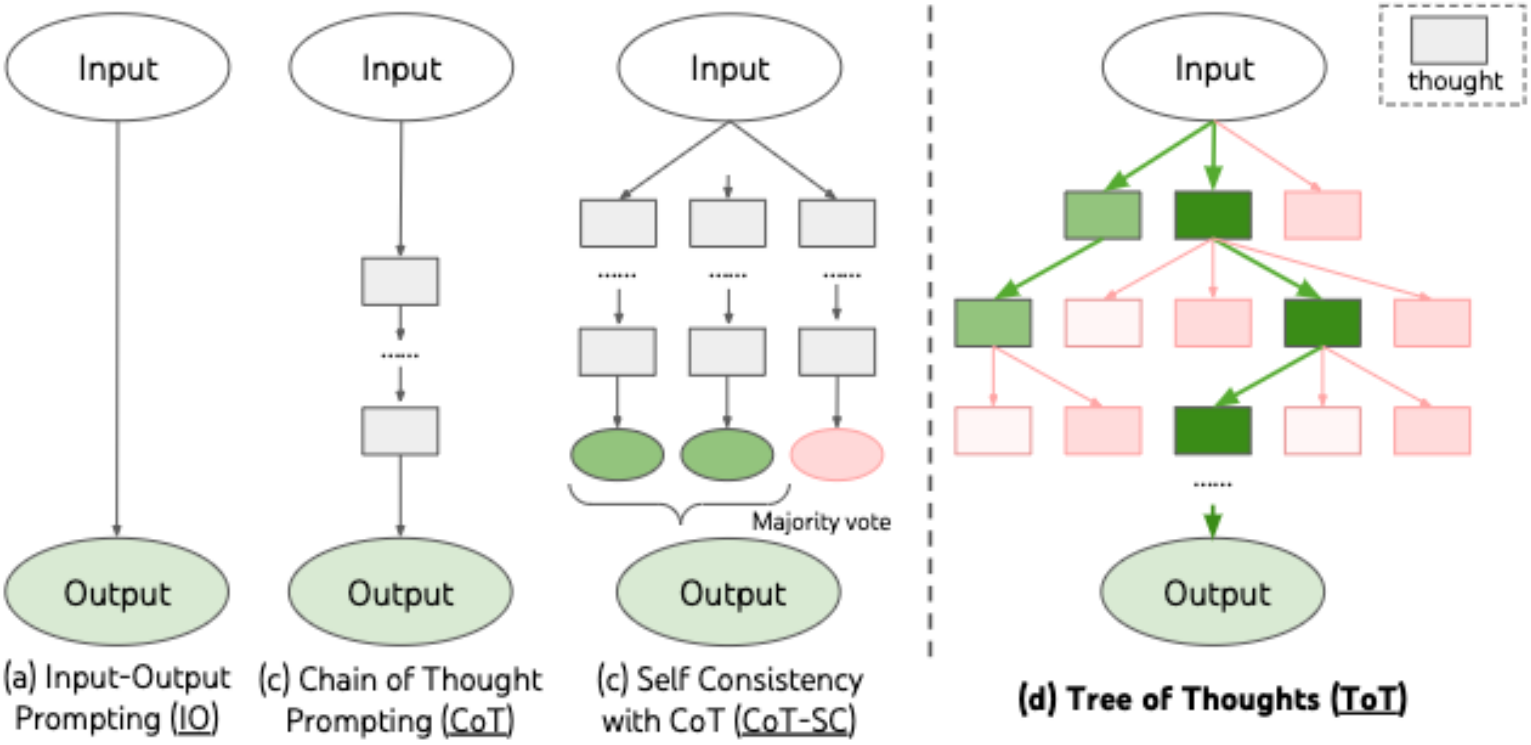
Back to chain of thought

CoT is linear - the LLM is expected to generate a reasoning path step-by-step

> *A genuine problem-solving process involves the repeated use of available information to initiate exploration, which discloses, in turn, more information until a way to attain the solution is finally discovered.*——*Newell et al.*

- "Tree of thought" allows the LLM to explore different branching paths, akin to search algorithms
- It allows LLMs to perform deliberate decision making through reasoning paths and self evaluation and even backtracking

Tree of thoughts (Yao et al., 2023)



Tree of thoughts

Instantiating ToT requires answering:

- Defining "thought decomposition". What is considered a thought? (needs to be "small" enough so can be diverse, and "big" enough so we can evaluate progress)

	Game of 24	Creative Writing	5x5 Crosswords
Input	4 numbers (4 9 10 13)	4 random sentences	10 clues (h1. presented;..)
Output	An equation to reach 24 (13-9)*(10-4)=24	A passage of 4 paragraphs ending in the 4 sentences	5x5 letters: SHOWN; WIRRA; AVAIL; ...
Thoughts	3 intermediate equations (13-9=4 (left 4,4,10); 10-4=6 (left 4,6); 4*6=24)	A short writing plan (1. Introduce a book that connects...)	Words to fill in for clues: (h1. shown; v5. naled; ...)
#ToT steps	3	1	5-10 (variable)

Table 1: Task overview. Input, output, thought examples are in blue.

Tree of thoughts

Instantiating ToT requires answering:

- Defining thought decomposition
- Thought generator - given a state in the tree, generate a new thought. For example: (a) sample from the LLM; (b) use a propose prompt
- State evaluator - given a frontier of states, evaluate the progress they make towards solving the problem (just like in search). For example: (a) value the state based on a prompt; (b) vote for the best state (so comparison among different states)
- Search algorithm - Breadth-first search (BFS); Depth-first search (DFS)

Note that ToT is problem specific, less generic, need to instantiate it

Results

Method	Success
IO prompt	7.3%
CoT prompt	4.0%
CoT-SC (k=100)	9.0%
ToT (ours) (b=1)	45%
ToT (ours) (b=5)	**74%**
IO + Refine (k=10)	27%
IO (best of 100)	33%
CoT (best of 100)	49%

Table 2: Game of 24 Results.

In Game of 24, you are given 4 numbers and basic arithmetic operations $(+ - */)$ and the goal is to obtain the number 24 by using all the numbers (each once)
For example, given "4 9 10 13" a solution could be
$(10 - 4) * (13 - 9) = 24$

Results

Method	Success Rate (%)		
	Letter	Word	Game
IO	38.7	14	0
CoT	40.6	15.6	1
ToT (ours)	**78**	**60**	**20**
+best state	82.4	67.5	35
-prune	65.4	41.5	5
-backtrack	54.6	20	5

Table 3: Mini Crosswords results.

Self-Reflection

Originally, self-refinement? (Madaan et al., 2023)

- The idea: Let the model "refine" its answer by providing *feedback* on its own answer
- Can do this in multiple steps
- Leads to a form of reasoning, or "self-reflection"

The Basic Algorithm

Algorithm 1 SELF-REFINE algorithm

Require: input x, model $\mathcal{M}$, prompts $\{p_{\text{gen}}, p_{\text{fb}}, p_{\text{refine}}\}$, stop condition $\text{stop}(\cdot)$

```
y_0 = M(p_gen || x)                                   ▷ Initial generation (Eqn. 1)
for iteration t ∈ 0, 1, ... do
    fb_t = M(p_fb || x || y_t)                        ▷ Feedback (Eqn. 2)
    if stop(fb_t, t) then                             ▷ Stop condition
        break
    else
        y_{t+1} = M(p_refine || x || y_0 || fb_0 || ... || y_t || fb_t)   ▷ Refine (Eqn. 4)
    end if
end for
return y_t
```

Food for thought: When to stop?

The Basic Algorithm

Algorithm 1 SELF-REFINE algorithm

Require: input x, model $\mathcal{M}$, prompts $\{p_{\text{gen}}, p_{\text{fb}}, p_{\text{refine}}\}$, stop condition $\text{stop}(\cdot)$

$y_0 = \mathcal{M}(p_{\text{gen}} \| x)$ ▷ Initial generation (Eqn. 1)
for iteration $t \in 0, 1, \ldots$ **do**
  $fb_t = \mathcal{M}\left(p_{\text{fb}} \| x \| y_t\right)$ ▷ Feedback (Eqn. 2)
  if $\text{stop}(fb_t, t)$ **then** ▷ Stop condition
    break
  else
    $y_{t+1} = \mathcal{M}\left(p_{\text{refine}} \| x \| y_0 \| fb_0 \| \ldots \| y_t \| fb_t\right)$ ▷ Refine (Eqn. 4)
  end if
end for
return y_t

Food for thought: When to stop? "... either stops at a specified timestep *t* or extracts a stopping indicator (e.g. a scalar stop score) from the feedback."

Example (Madaan et al., 2023)

(a) **Dialogue:** x, $\mathbf{y_t}$

```
User: I am interested
in playing Table
tennis.

Response: I'm sure
it's a great way to
socialize, stay active
```

(b) **FEEDBACK** $\mathbf{fb}$

```
Engaging: Provides no
information about table
tennis or how to play it.

User understanding: Lacks
understanding of user's
needs and state of mind.
```

(c) **REFINE** $\mathbf{y_{t+1}}$

```
Response (refined): That's
great to hear (...) ! It's
a fun sport requiring
quick reflexes and good
hand-eye coordination.
Have you played before, or
are you looking to learn?
```

(d) **Code optimization:** x, $\mathbf{y_t}$

```
Generate sum of 1, ..., N
def sum(n):
    res = 0
    for i in range(n+1):
        res += i
    return res
```

(e) **FEEDBACK** $\mathbf{fb}$

```
This code is slow as
it uses brute force.
A better approach is
to use the formula
... (n(n+1))/2.
```

(f) **REFINE** $\mathbf{y_{t+1}}$

```
Code (refined)

def sum_faster(n):
  return (n*(n+1))//2
```

Results (Madaan et al., 2023)

	GPT-3.5		ChatGPT		GPT-4	
Task	Base	+SELF-REFINE	Base	+SELF-REFINE	Base	+SELF-REFINE
Sentiment Reversal	8.8	**30.4** (↑21.6)	11.4	**43.2** (↑31.8)	3.8	**36.2** (↑32.4)
Dialogue Response	36.4	**63.6** (↑27.2)	40.1	**59.9** (↑19.8)	25.4	**74.6** (↑49.2)
Code Optimization	14.8	**23.0** (↑8.2)	23.9	**27.5** (↑3.6)	27.3	**36.0** (↑8.7)
Code Readability	37.4	**51.3** (↑13.9)	27.7	**63.1** (↑35.4)	27.4	**56.2** (↑28.8)
Math Reasoning	**64.1**	**64.1** (0)	74.8	**75.0** (↑0.2)	92.9	**93.1** (↑0.2)
Acronym Generation	41.6	**56.4** (↑14.8)	27.2	**37.2** (↑10.0)	30.4	**56.0** (↑25.6)
Constrained Generation	28.0	**37.0** (↑9.0)	44.0	**67.0** (↑23.0)	15.0	**45.0** (↑30.0)

Table 1: SELF-REFINE results on various tasks using GPT-3.5, ChatGPT, and GPT-4 as base LLM. SELF-REFINE consistently improves LLM. Metrics used for these tasks are defined in Section 3.2.

About Feedback (Madaan et al., 2023)

- The model needs to be prompted to write feedback that is *actionable* and *specific* (via $fb^{(k)}$)
- Actionable: concrete action that would improve the output; Specific: identify concrete phrases to change

Example for code reasoning:

- Good: *Avoid repeated calculations in the for loop*
- Bad: *Improve the efficiency of the code*

Feedback Improvements

Task	y_0	y_1	y_2	y_3
Code Opt.	22.0	27.0	27.9	**28.8**
Sentiment Rev.	33.9	34.9	36.1	**36.8**
Constrained Gen.	29.0	40.3	46.7	**49.7**

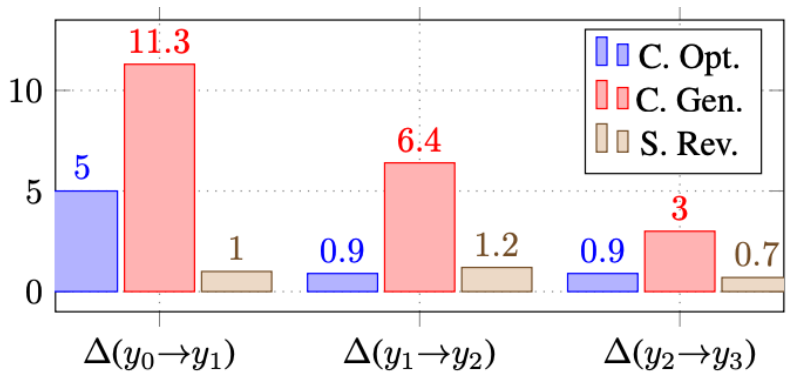


Figure 4: **Left**: Iteration-wise score improvements. Early iterations significantly improve output quality, and scores generally keep improving with more iterations. **Right**: SELF-REFINE Performance improvements with iterations. Most gains(Δ) are in the initial iterations for both Code Opt. and Sentiment Reversal. The numbers are averaged over ChatGPT, GPT-3.5, and GPT-4. Task abbreviations: C. Opt. (Code Optimiz.), S. Rev. (Sentiment Reversal), C. Gen. (Constrained Generation).

Using External Verifiers/Solvers

- The idea: use an external tool to solve a problem/question
- Breaks the boundaries of LLM-only modelling, but is a decoding-only method in most cases

Example: Mathematical Reasoning using Lean

What is Lean?

- Lean is a theorem prover assistant, originally used by mathematicians to improve their work process
- It is open-source with a vibrant community support and a very large base of theorems and lines of code

A Mini-crash Course in Lean

- Lean verifies a proof through "type checking"
- There are so-called *terms* in Lean which have a *type*
- By applying *tactics*, Lean tries to create terms based on other terms. If at any point, the type of the term is wrong, then Lean gives an error, implying the proof is incorrect
- If you are familiar with λ expressions, think of Lean as compiling a "proof" into a λ expression which is "fully executed" only if the proof follows correct reasoning

A Mini-crash Course in Lean

```
theorem ex (p q r : Prop) : (p → q) → (q → r) → p → r := by
  intro hpq hqr hp
  have hq : q := hpq hp
  have hr : r := hqr hq
  exact hr
```

This is a Lean proof for the statement
$(p \to q) \to (q \to r) \to p \to r$

- The arrows denote functions (so $p \to q$ is a function that takes a proposition and returns another) - but it is used here as implication too!
- There is always a "goal" in the background - it is a type. In the beginning it is just the full statement above, we have to get the goal to be of that type
- The intro command introduces three new terms, peeling off the arrows (h for "hypothesized") and making the new goal be r

A Mini-crash Course in Lean

```
theorem ex (p q r : Prop) : (p → q) → (q → r) → p → r := by
  intro hpq hqr hp
  have hq : q := hpq hp
  have hr : r := hqr hq
  exact hr
```

...

- The have commands now validate each of the first two terms in the overall implication statement (hq, for example, is validated, by stating roughly, apply the term hpq on hp, and check whether it is of type q)
- exact hr checks that hr has the type of the goal, which due to the intro statement is r
- Note the proof is semi "constructive"
- Note we had to write the full proof! The proof is not automatically generated, just verified

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)
then another term is added $hpq\colon q \to r$ and the goal becomes $p \to r$.
The context become $hpq : p \to q, hqr : q \to r$

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)
then another term is added $hpq\colon q \to r$ and the goal becomes $p \to r$.
The context become $hpq : p \to q, hqr : q \to r$
then another term is added $hp\colon p$ and the goal becomes r The context become $hpq : p \to q, hqr : q \to r, hp : p$

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)
then another term is added $hpq\colon q \to r$ and the goal becomes $p \to r$.
The context become $hpq : p \to q, hqr : q \to r$
then another term is added $hp\colon p$ and the goal becomes r The context become $hpq : p \to q, hqr : q \to r, hp : p$
then, $have\ hq : q := hpq\ hp$ creates a new term $hq : q$. The context becomes $hpq : p \to q, hqr : q \to r, hp : p, hq : q$

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)
then another term is added $hpq\colon q \to r$ and the goal becomes $p \to r$.
The context become $hpq : p \to q, hqr : q \to r$
then another term is added $hp\colon p$ and the goal becomes r The context become $hpq : p \to q, hqr : q \to r, hp : p$
then, $have\ hq : q := hpq\ hp$ creates a new term $hq : q$. The context becomes $hpq : p \to q, hqr : q \to r, hp : p, hq : q$
then, $have\ hr : r := hpr\ hq$ creates a new term $hq : q$. The context becomes $hpq : p \to q, hqr : q \to r, hp : p, hq : q, hr : r$

Lean Example - State is Context with Goals

Want to prove: $(p \to q) \to (q \to r) \to p \to r$. That's the goal, and that's the state with that goal ("context" is empty).
The *intro hpq hqr hp* introduces into the state the terms by peeling off the implications: first, $hpq\colon p \to q$ (the goal becomes $(q \to r) \to p \to r$)
then another term is added $hpq\colon q \to r$ and the goal becomes $p \to r$.
The context become $hpq : p \to q, hqr : q \to r$
then another term is added $hp\colon p$ and the goal becomes r The context become $hpq : p \to q, hqr : q \to r, hp : p$
then, *have* $hq : q := hpq\, hp$ creates a new term $hq : q$. The context becomes $hpq : p \to q, hqr : q \to r, hp : p, hq : q$
then, *have* $hr : r := hpr\, hq$ creates a new term $hq : q$. The context becomes $hpq : p \to q, hqr : q \to r, hp : p, hq : q, hr : r$
then, *exact hr* finishes the goal which was r.

A Simpler Case

John had 5 apple. He ate 2 of them. How many does he have left?

```
theorem apples : 5 - 2 = 3 := by rfl
```

- Here, rfl is a way to tell Lean to type check "apples" by ensuring that the proposition 5-2=3 is true.
- The proposition is of a special type that rfl handles a=b. It indeed follows the type checking only if a indeed equals b
- The important point: We provided the solution here, 3! We did not try to solve it.

How is that Related to LLMs?

- Say we have a prompt p, we run it through the LLM with chain of thought
- We then ask the LLM to convert the chain of thought into a Lean formalisation
- We then verify the Lean formalisation to check whether it "type checks"
- If it is, we return that CoT with the answer
- If not, we "do something about it"

Food for thought: Does the third step passing correctly always implies the answer is correct?

How is that Related to LLMs?

- Say we have a prompt p, we run it through the LLM with chain of thought
- We then ask the LLM to convert the chain of thought into a Lean formalisation
- We then verify the Lean formalisation to check whether it "type checks"
- If it is, we return that CoT with the answer
- If not, we "do something about it"

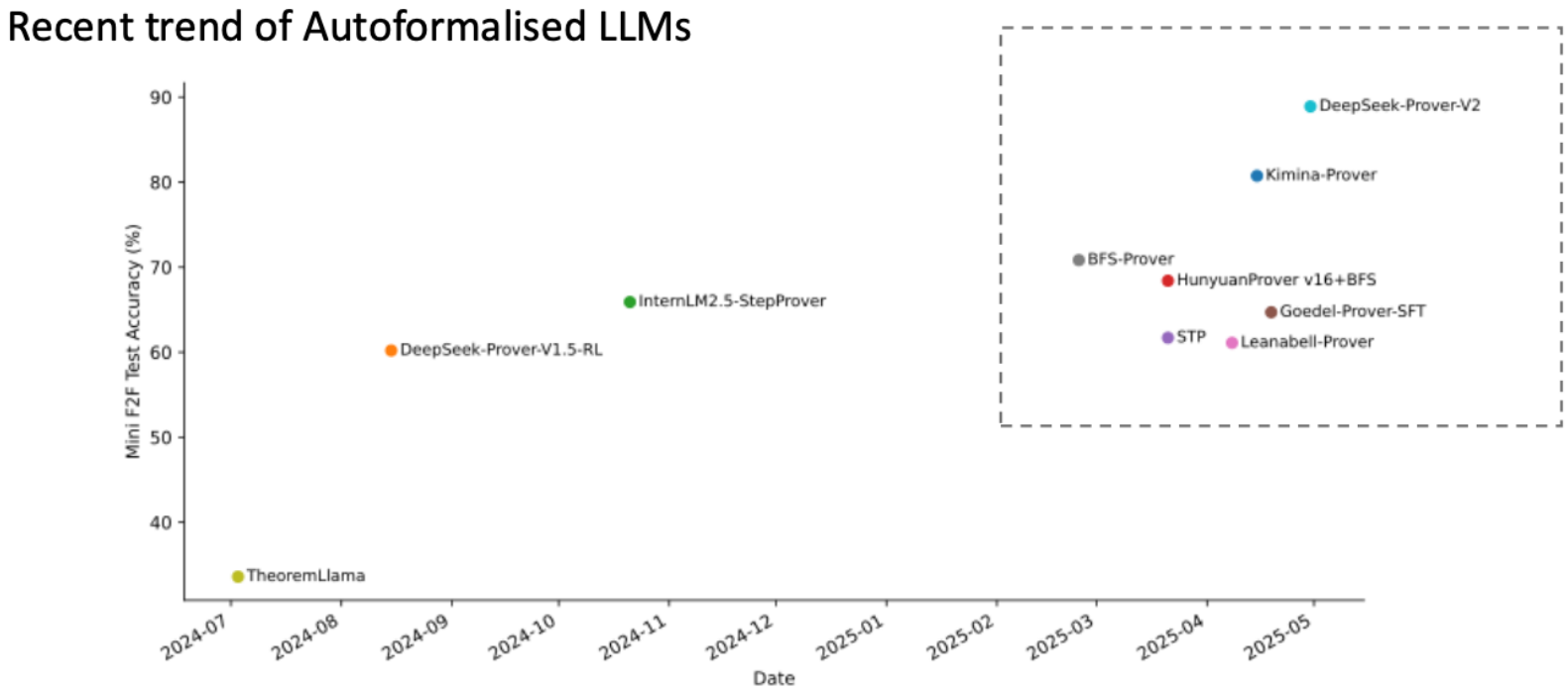
Food for thought: Does the third step passing correctly always implies the answer is correct? No, because the Lean formalisation itself could be wrong

Types of Errors with the Lean Formalisation

There can be several types of errors:

- The formalised Lean code doesn't "compile" (doesn't follow Lean syntax or proper semantics)
- The formalised Lean code doesn't properly represent the prompt p, but type checks
- The formalised Lean code represents the prompt, but doesn't type check (reasoning is incorrect)

Some Results



MiniF2F is a mathematical reasoning dataset, with olympiad-level questions

"Prompt engineering"

Sometimes to get better so-called "reasoning chains", what is required is better/specific prompting

You will explore that in the assignment with a specific way of decomposing and formalising simple mathematical questions (Ong et al., 2025)

A "taste" of this

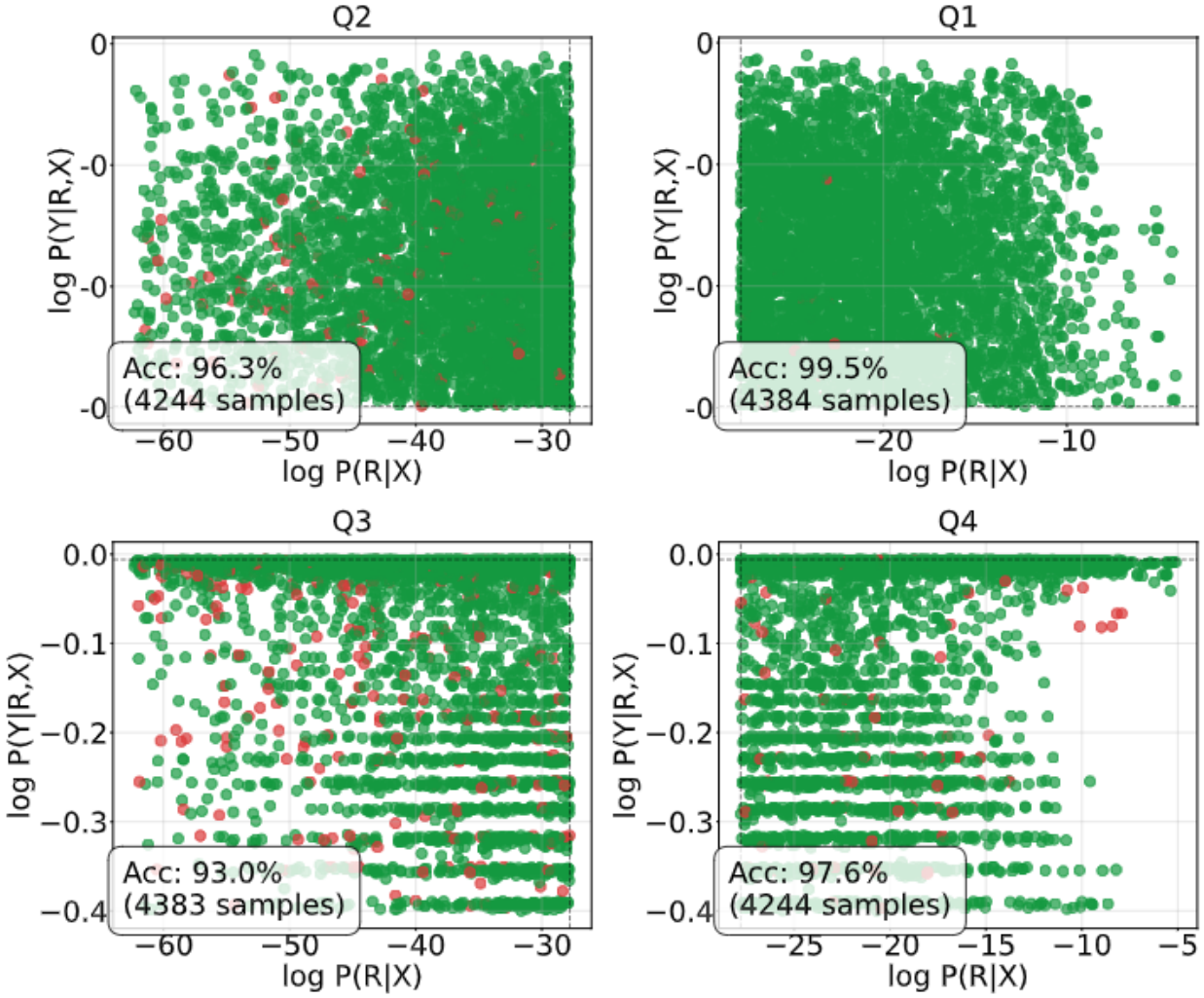
In your assignment, you will see how LLMs can reason and solve simple highschool-type questions

You will be asked to use the ELM system of the University. If you have not yet received a code and instructions, please let me know as soon as possible

We allow longer time to solve the assignment this year

Allow the LLMs to "reason" *about the assignment* only as specified in the instructions...

PiCSAR, Ong et al. (2025)



When models are confident about the reasoning chain and their final answer, their accuracy is high

Language reasoning models

We focused mostly on inference. LRMs are LLMs that are trained to:

- break problems into intermediate steps
- track variables, constraints, or facts
- perform logical, mathematical, or symbolic reasoning

Examples: GPT-4, GPT-4o, OpenAI o1, OpenAI o3, Minerva, PaLM-2-Math, Gemini 1.5 Pro, Claude 3 Opus, AlphaCode, AlphaCode 2, DeepSeek-R1, Qwen-QwQ, LLaMA-2-Reasoning, LLaMA-3-Instruct, Mistral Large, Mixtral-8x22B-Instruct, ReAct-augmented LLMs, Program-of-Thought models, ...

Rely on large amounts of "reasoning" data

Summary

Reasoning in LLMs often refers to a process of deliberation that the LLM follows to reach an answer. We covered:

- Stepwise: chain-of-thought, self-consistency, tree-of-thoughts...
- Self-reflection
- Using external solvers